
Coordination, Trustworthiness, Sovereignty and Performance Assurance

Hong-Linh Truong
linh.truong@aalto.fi



30.09.2026



Content is available under
CC BY-SA 4.0 unless otherwise stated

Learning objectives

- Understand concepts of coordination and related trustworthiness, sovereignty, and performance
- Understand key aspects in coordination for HIS
- Understand and able to design and apply coordination techniques for HIS

Content

- Overview of coordination for HIS
- Capability discovery, monitoring and observability
- Coordination styles and architectures

Recall: Hybrid Intelligence Systems (HIS) for solving complex problems

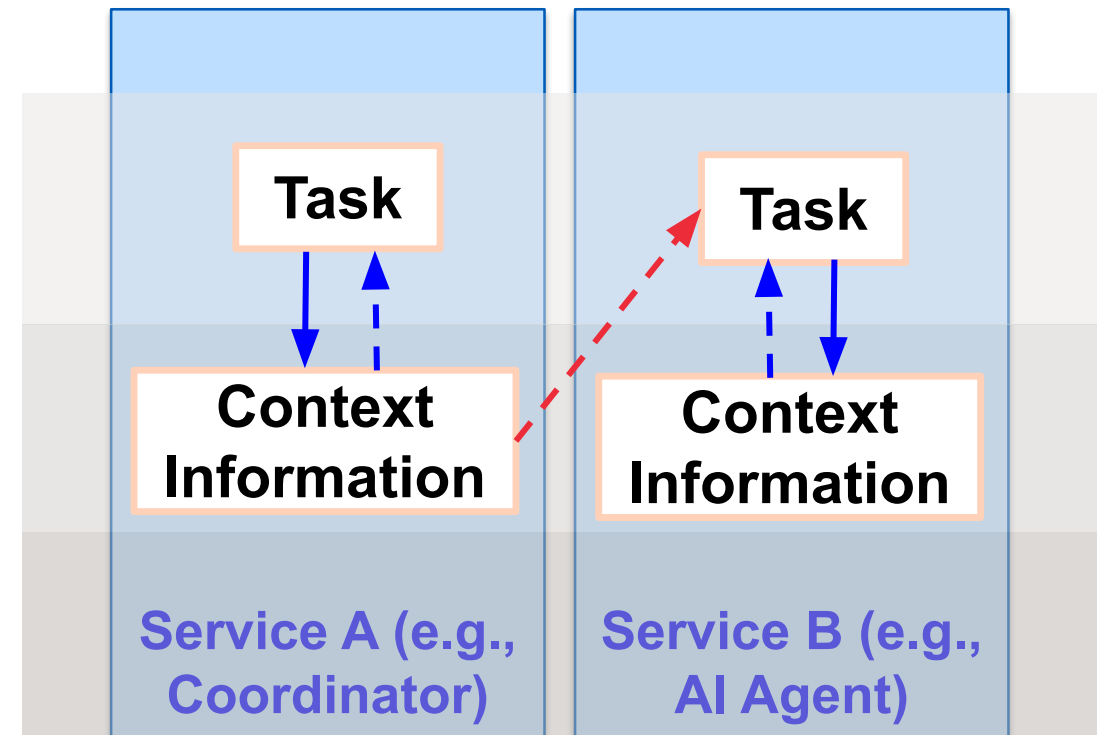
- HIS has different types of components
 - humans, traditional AI/ML services, LLM-based services, AI agents and other data/compute services
- These types of components
 - diverse functions and operational environments
 - different behaviors/characteristics

Together they solve problems by completing (dependent) tasks/works

Types of context information for service operations

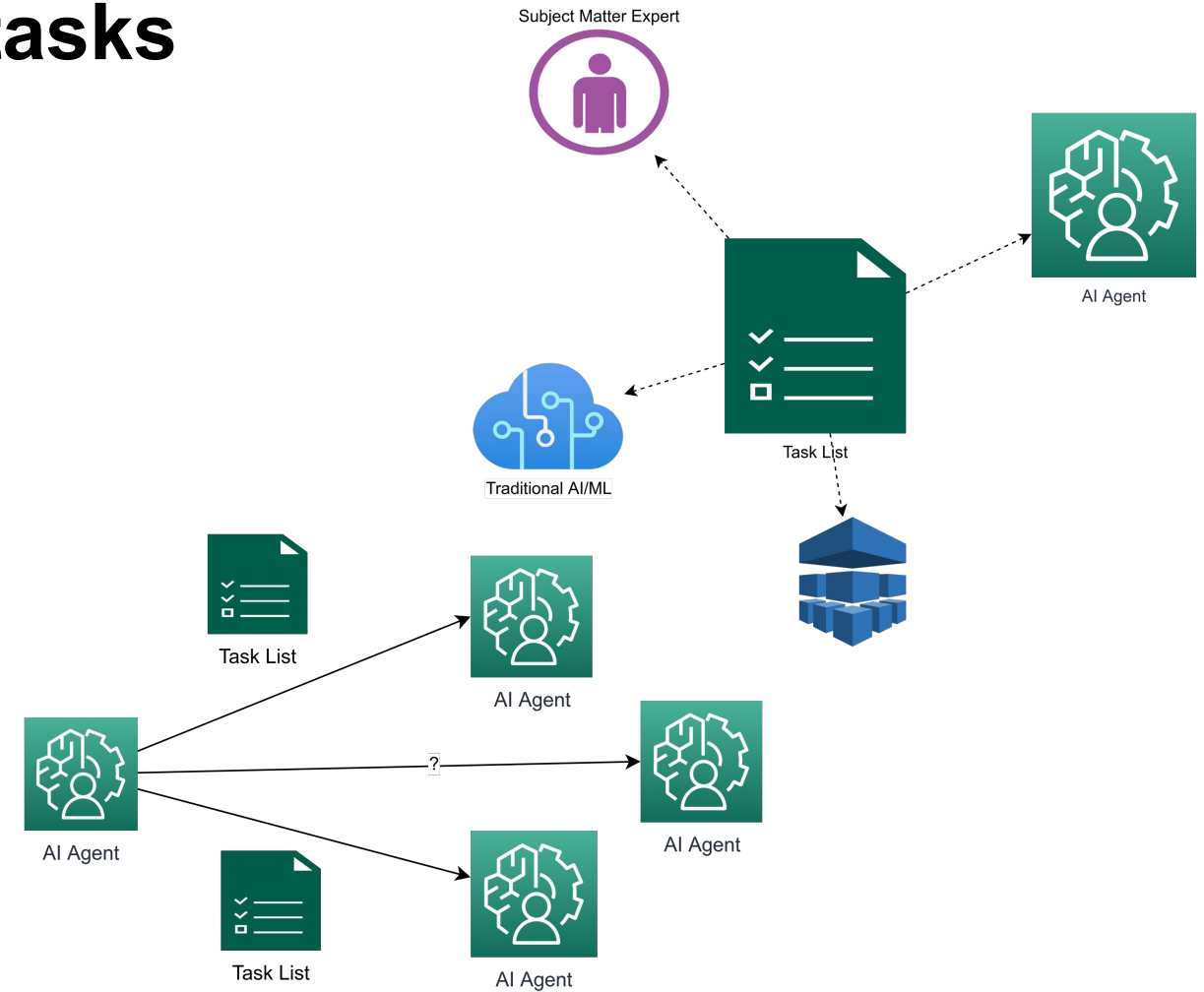
- A service has different types of information for its operation
 - static configurations
 - dynamic runtime state information, short term persistent data per session
 - long-term cross-session, persistent information
- Some information must be shared among services for operations

Abstract view from task and context information sharing



Working together to complete tasks

- Tasks can be defined for application-specific problems but also for HIS operations
- Stateless vs stateful, long runtime tasks (e.g., for humans and AI agents)
- Different services have different ways of share contexts/historical contents.



If there is a list of tasks, how are tasks distributed?

The problems

Dynamicity of tasks & required capabilities

Complexity of flows of task executions and quality

Diversity of capabilities

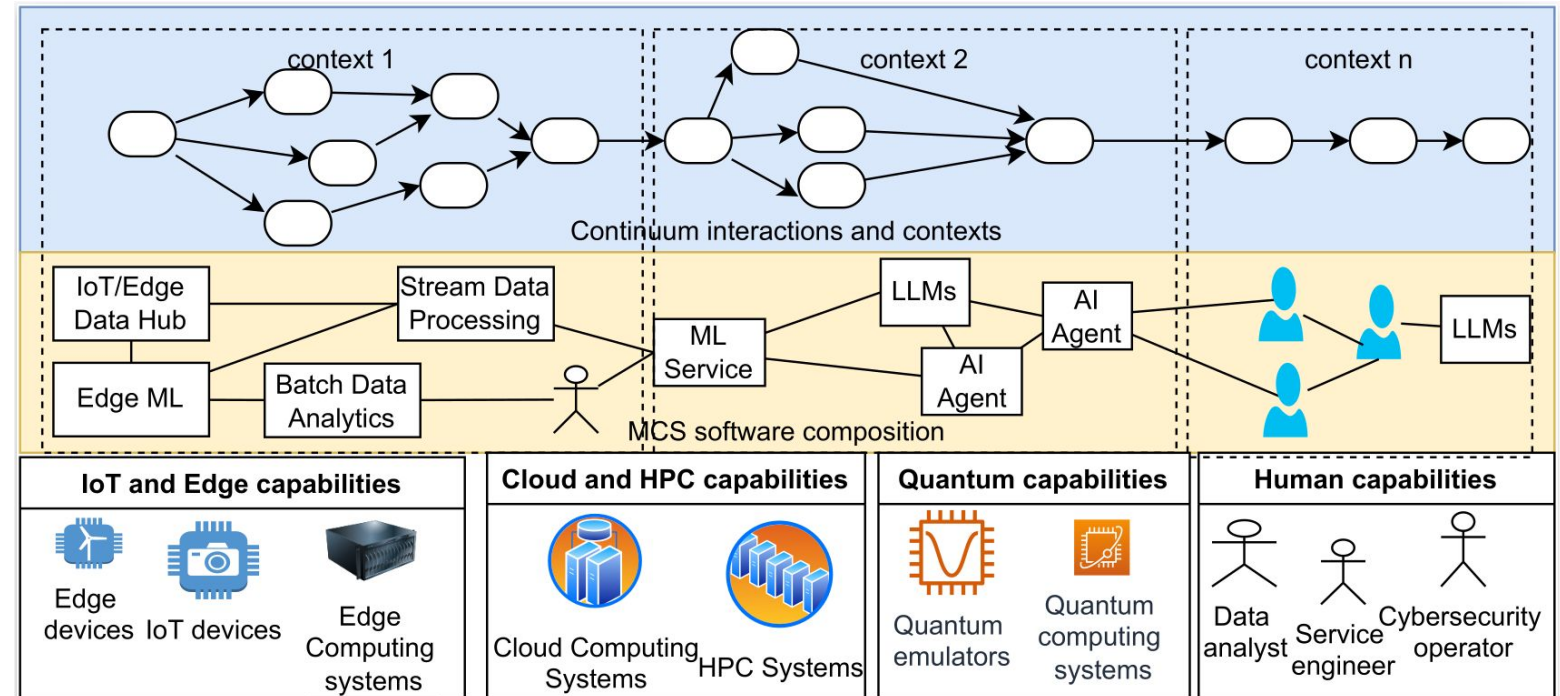


Figure source: Hong-Linh Truong, “*New Frontiers in Service Engineering Analytics for Multi-Continuum Systems (FrontSEA)*”, working paper, 2025

At runtime, coordination must make sure services/participants completing (dependent) tasks/works, solving required problems

Coordination objectives: functional vs non-functional requirements

- Functional requirements are about HIS capabilities/functions
- Non-functional requirements
 - trustworthiness: safety, explainability, accountability, etc.
 - sovereignty: able to control and own data and service
 - performance: service and data quality, time, cost, etc.
- Elasticity and resilience are techniques for supporting functional and non-functional requirements
 - e.g., replace a failed service by another one to assure the function operable
 - increase more agents to improve the performance
 - remove an agent violating data sovereignty without compromising the quality

NIST: Characteristics of trustworthiness in AI

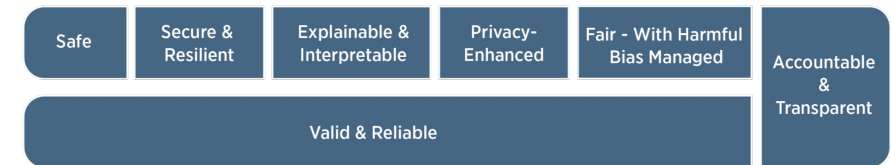


Figure source:

<https://airc.nist.gov/airmf-resources/airmf/3-sec-characteristics/>

Coordination follows multi-continuum dimensions

- Computing
 - scale and control data processing
 - dynamic serving: manage loads and services
- Time
 - between different sessions and overtaking
- Intelligence
 - switch and combine human and AI, elasticity of capabilities

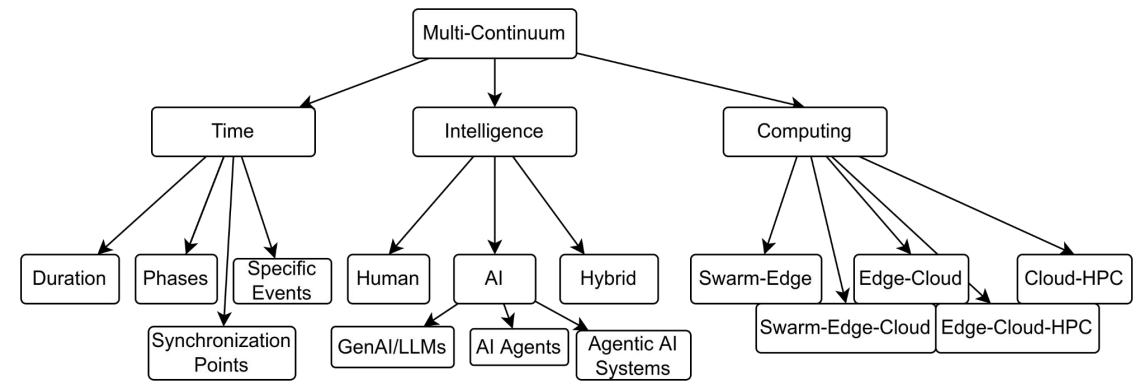


Figure source: Hong-Linh Truong, Kostas Magoutis, “Multi-continuum view for Swarm-Edge-Cloud Service-based Applications”, **working paper**, 2025

Runtime dynamicity: so it is not a fix integration!
Trustworthiness, sovereignty, performance, etc. must be guaranteed

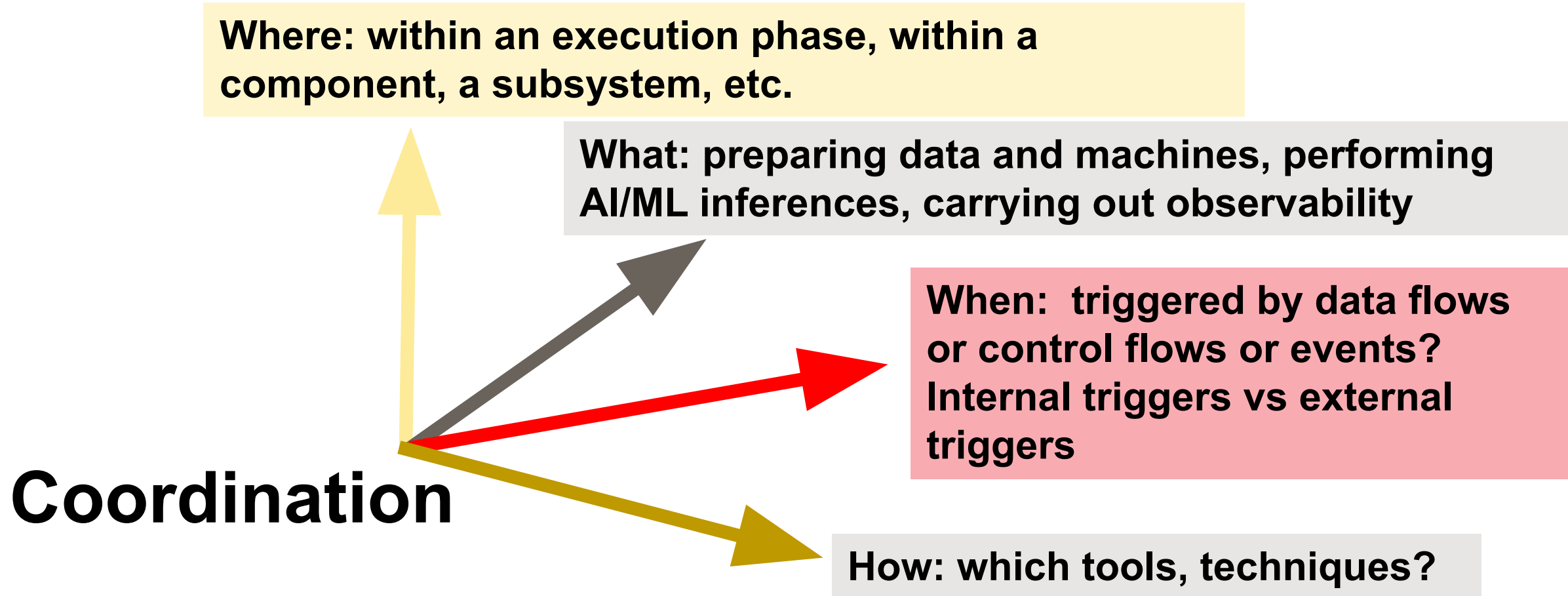
How do we know the availability of capabilities and other important factors

- Capability discovery and registry
 - common service discovery techniques in service-oriented computing
 - emerging solutions for AI agents:
 - A2A agent card
 - Agentic Resource Discovery Specification (ARD):
<https://agenticresourcediscovery.org/>,
 - Other services:
 - <https://github.com/agentnameservice/ans>,
 - <https://github.com/huggingface/hf-discover>
- *The first line of assurance for trustworthiness, sovereignty and performance*
 - protection from malicious descriptions
 - check and evaluate properties of services
 - profiling, scenario-based experimentation, benchmarking and verification
 - continuously reassessment of existing capabilities and services offering them

How do we know the current situation

- Monitoring, observability and prediction
 - logging, tracing, anomaly detection of infrastructures, consumer requests and service/data tasks
- *The second line of assurance for trustworthiness, sovereignty and performance*
 - verifying capabilities before using them and during the utilization
 - based on runtime monitoring, observability and
 - near real time measurement and metric analysis
 - quality and access control guards

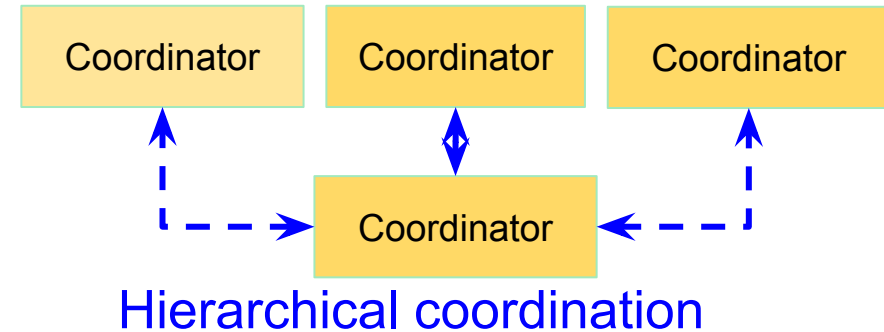
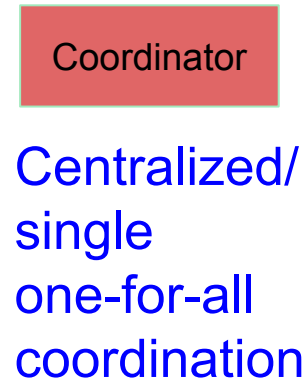
W3H: what, when, where and how for coordination



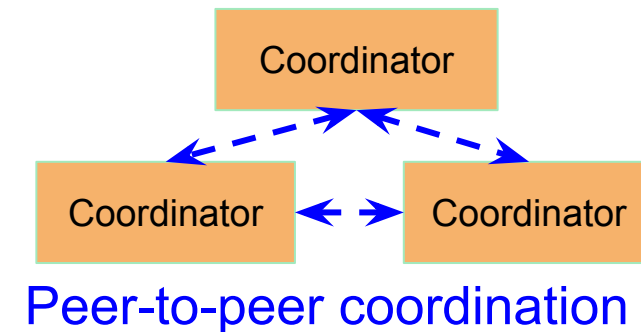
Different coordination architectures

Possible coordination architectures

**Coordinator
performs the
coordination**



Services offering capabilities are
from distributed, convergence
infrastructures
⇒ *system of system coordination*



Runtime operations during the coordination

- Operations during runtime coordination
 - plan tasks, select suitable services for task execution, dispatch tasks, control the execution of tasks, and monitoring task status
- *The third line of assurance for trustworthiness, sovereignty and performance by coordinator*
 - runtime capability selection
 - runtime guards, adaptive decision of task execution
 - detailed observability and explanations

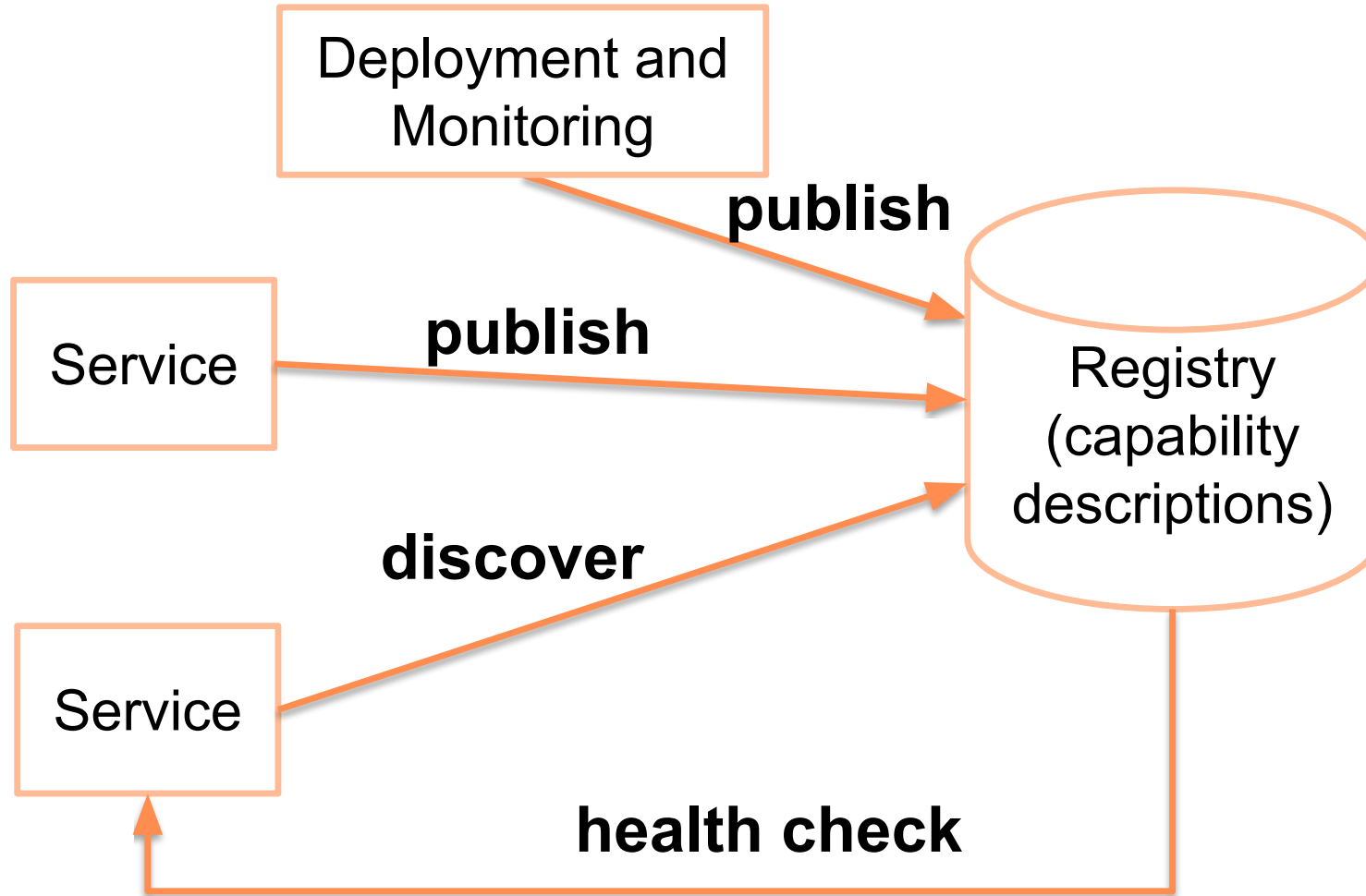
Capability discovery, monitoring, and observability

Service state management

- Service information
 - include states and other important configuration information
 - many instances
 - cross different infrastructures/data centers
- Related components
 - services themselves
 - monitoring component, deployment component, orchestration controllers
- Lifecycle: very dynamic in elastic environments
 - start, run, shutdown, restart, scale

Registry and discovery mechanisms enable the search of capabilities

Service Discovery principle



- **Key requirements**

- fast
- reliable
- consistent
- secure
- cross data centers
- simple APIs

But what about for HIS?

- Different types of components
 - humans, AI agents, LLM-based services, etc.
 - need different descriptions, maybe different syntaxes
- Some approaches
 - pre-defined mapping: e.g., using consistent hashing or **global** namespace
 - discover **relevant services** and then **ask** relevant services about capabilities
 - use a **dedicated data discovery service** and ask the service for capabilities

AgentCard: description in the A2A protocol

<https://agent2agent.info/docs/concepts/agentcard/#agentcard-structure>

Discovery of MCP server and then discovery of MCP offerings

Official MCP Registry

Discover Model Context Protocol servers

<https://registry.modelcontextprotocol.io/>

Open Agent Specs

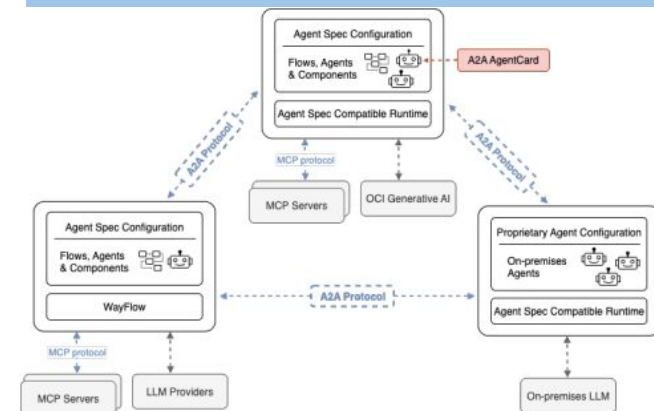
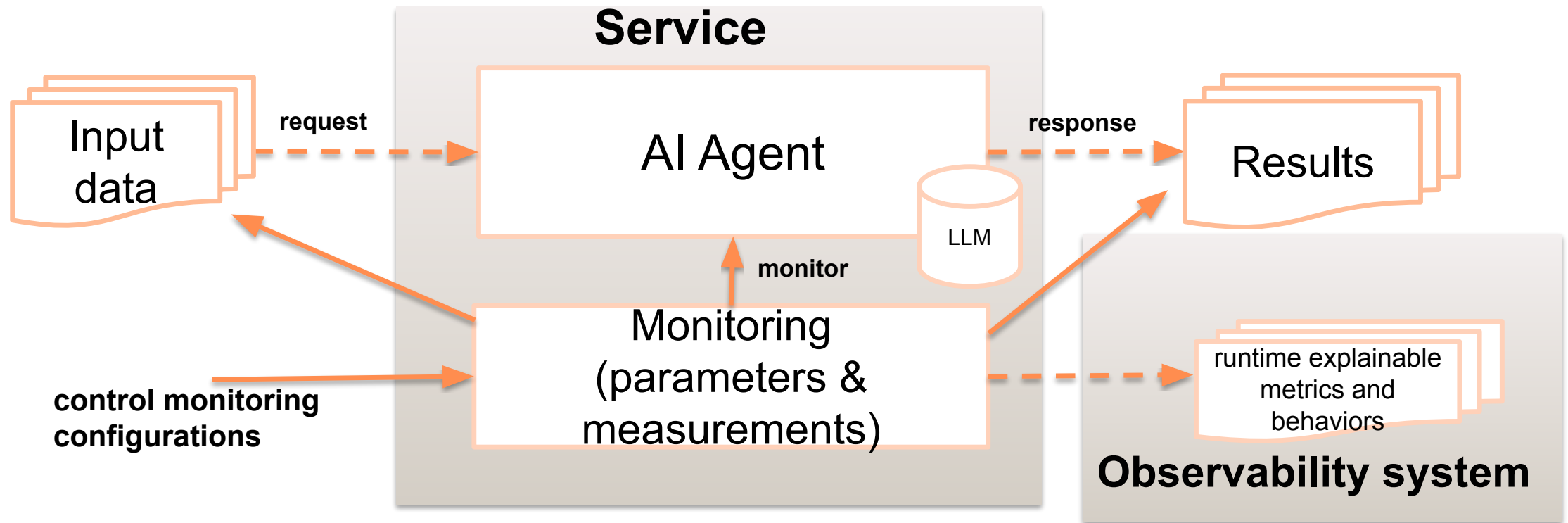


Figure source: Amini et al., Open Agent Specification: Enabling Cross-Framework Comparison of AI Agents. <https://doi.org/10.1145/3786335.3813130>

Open source: <https://github.com/oracle/agent-spec>

Monitoring and observability

E.g., capture important metrics (especially HHS emerging metrics)



Example: detecting/profiling elasticity

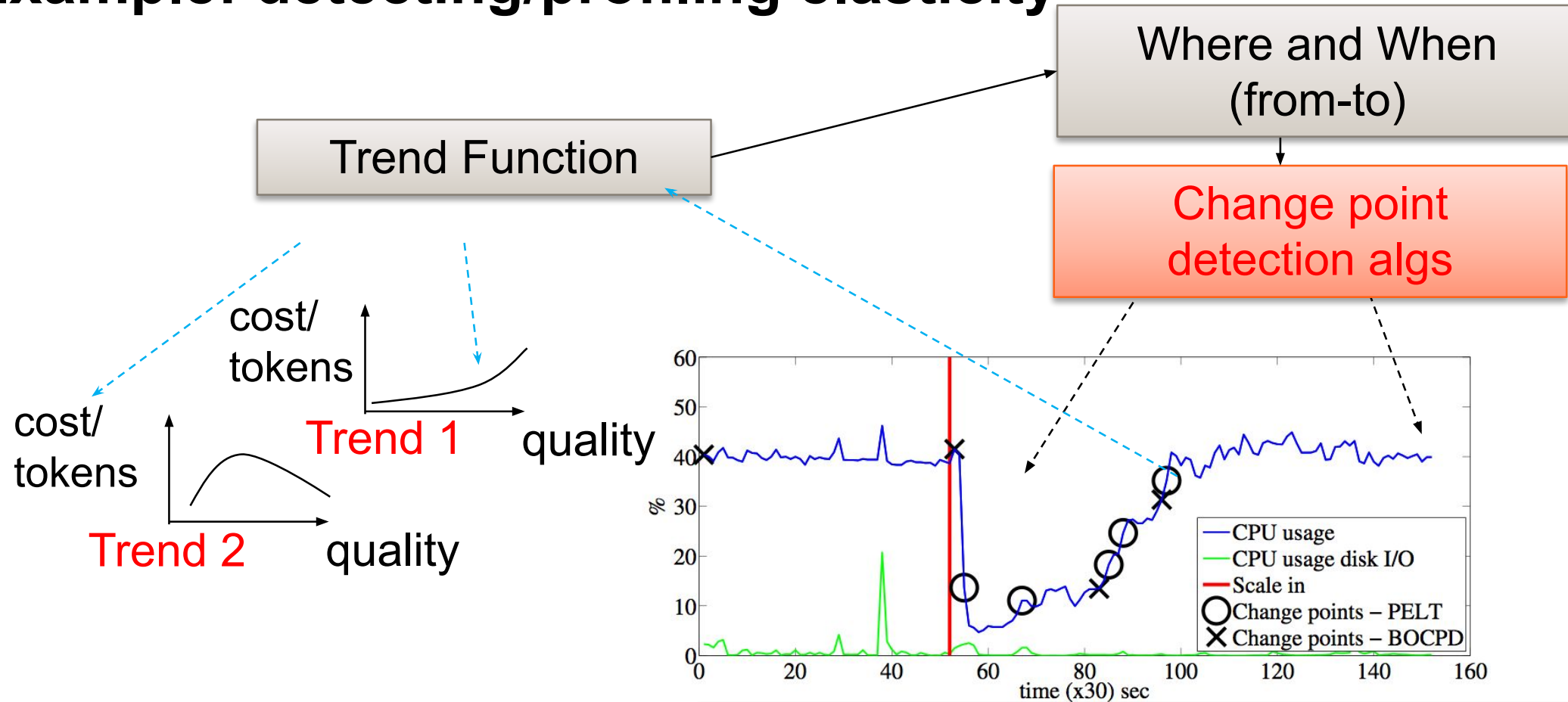
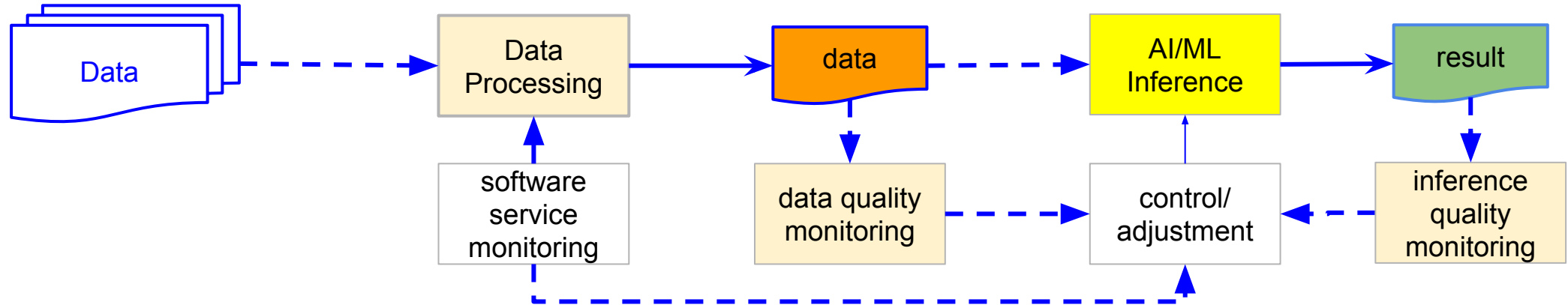


Figure source: Alessio Gambi et al., "On estimating actuation delays in elastic computing systems". SEAMS 2013: 33-42

Known techniques for anomaly detection and forecasting can be applied

Example: detecting quality of intermediate/final data or inference results, dealing with robustness/reliability in data

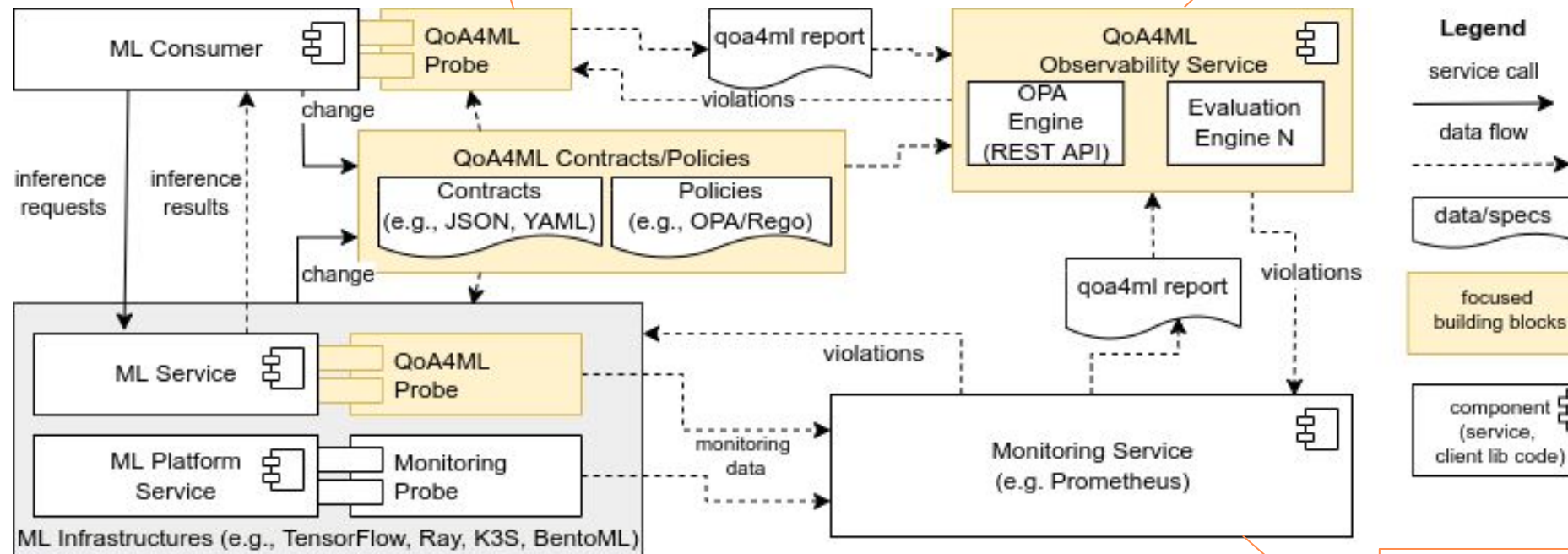


- Need different tools/techniques to evaluate quality of data
 - profiling, sampling, drift detection
 - very tricky with streaming data: quality vs overhead
 - very different from software performance evaluation
- Subject Matter Expert (SME) integration: human-in-the-loop for quality

Example: monitoring utilities and observability service with QoA4ML

Monitor ML-specific attributes (and others)

Design for different engines to be used



Plugins, reuse well-known monitoring systems

Example: policies for monitoring violations

Changed at runtime and easily be combined with *other types of policies in Infrastructure-as-Code* in edge-cloud continuum

```
{
  "resources": {
    "mlmodels": [
      {
        "id": "ml_inference",
        "mlinfrastructures": "tensorflow",
        "machinetypes": [
          "edge", "cloud", "inferencemodes": "dynamic"
        ]
      }
    ],
    "quality": {
      "services": [
        {
          "ResponseTime": {
            "operators": "max",
            "unit": "s",
            "value": 0.05,
            "class": ["performance"],
            "machinetypes": ["edge"]
          }
        }
      ],
      "data": [
        {
          "Accuracy": {
            "operators": "min",
            "unit": "percentage",
            "value": 80,
            "class": ["qualityofdata"]
          }
        }
      ],
      "mlmodels": [
        {
          "Accuracy": {
            "operators": "min",
            "unit": "percentage",
            "value": 80,
            "class": ["Accuracy"],
            "machinetypes": ["edge"]
          }
        }
      ]
    }
  }
}
```



Rego based implementation

```
1 package qoa4ml.bts.alarm
2 import data.bts.contract as contract
3 default contract_violation = false
4 contract_violation = true {
5   count(violation) > 0
6 }
7 # The policy checker will receive the contract and the runtime information
8 # input variable: the input of runtime metrics
9 violation[[input.client_info, "Accuracy violation on edge resource"]]{
10   input.service_info.machinetypes == "edge"
11   input.service_info.metric[_] == "Accuracy"
12   some i, j
13   contract.quality.mlmodels[i].Accuracy.machinetypes[j] == "edge"
14   input.metric.Accuracy < contract.quality.mlmodels[i].Accuracy.value
15 }
16 violation[[input.client_info, "ResponseTime violation on edge resource"]]{
17   input.service_info.machinetypes == "edge"
18   input.service_info.metric[_] == "ResponseTime"
19   some i, j
20   contract.quality.services[i].ResponseTime.machinetypes[j] == "edge"
21   input.metric.ResponseTime > contract.quality.services[i].ResponseTime.value
22 }
23 violation[[input.client_info, "Data quality violation"]]{
24   input.service_info.metric[_] == "DataAccuracy"
25   some i
26   contract.quality.data[i].Accuracy.operators == "min"
27   input.metric.DataAccuracy < contract.quality.data[i].Accuracy.value
28 }
```

QoA4ML specs and prototype: <https://github.com/rdsea/QoA4ML>
"QoA4ML - A Framework for Supporting Contracts in Machine Learning Services,"
ICWS 2021 doi: 10.1109/ICWS53863.2021.00066.

Assurance for Trustworthiness, Sovereignty and Performance

- Guards and runtime policies for service utilization
 - data and compute sovereignty (e.g., location and type of services)
 - applied to trajectory-level invocation paths
- Data quality evaluation
 - where: input, within, and output of the service
 - what: syntax- vs semantic-level quality evaluation for domain-specific data
 - E.g., guardrails for AI by using different validators
(<https://guardrailsai.com/guardrails/docs/concepts/validators>)
- Performance monitoring
 - tokens/cost, time (service/latency), failures, etc.

Coordination styles and architectures

Coordination styles (how)

- Direct calls
 - based on known service descriptions
- Orchestration
 - task graphs and dependencies are based on control or data flows
 - dedicated orchestrator
 - tasks triggered based on completeness of other tasks or the availability of data
 - often implemented as workflows
 - current AI agent frameworks tend to have workflows implemented as code
- Reactiveness/choreography
 - follow a reactive model
 - tasks are reacted/triggered based on messages

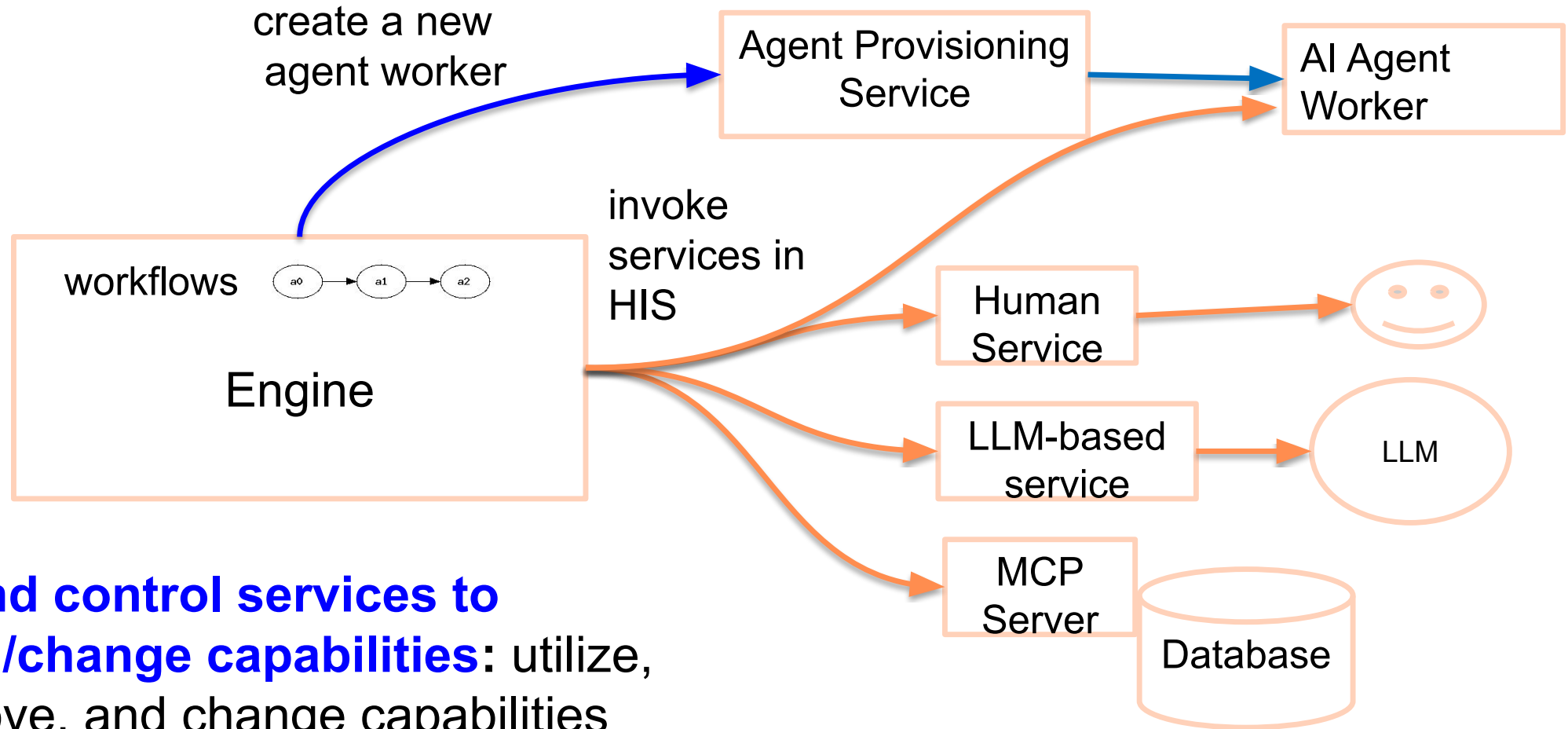
Direct call

- Call: must know how to call a service
 - service APIs, service descriptions
 - AI Agent: A2A protocol, Agent Card
 - Service API to human inputs: suitable human interfaces (UI, CLI, messaging systems)
- Direct call can also deal with various context information
 - static configurations
 - dynamic runtime state information
 - long-term cross-session, persistent information

Coordination using the orchestration architectural style (1)

- Using workflow concepts and technologies
 - architectures are well-known, leverage many types of services and cloud technologies
- Required components
 - workflow/pipeline specifications/languages (also UI)
 - orchestration engines
- Execution environments
 - cloud platforms (e.g., VMs, containers, Kubernetes)
 - heterogeneous computing resources (HPC, servers, AI Factories etc.)

Coordination using the orchestration architectural style (2)

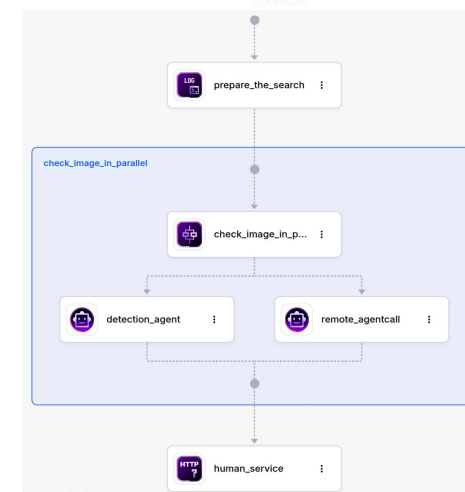
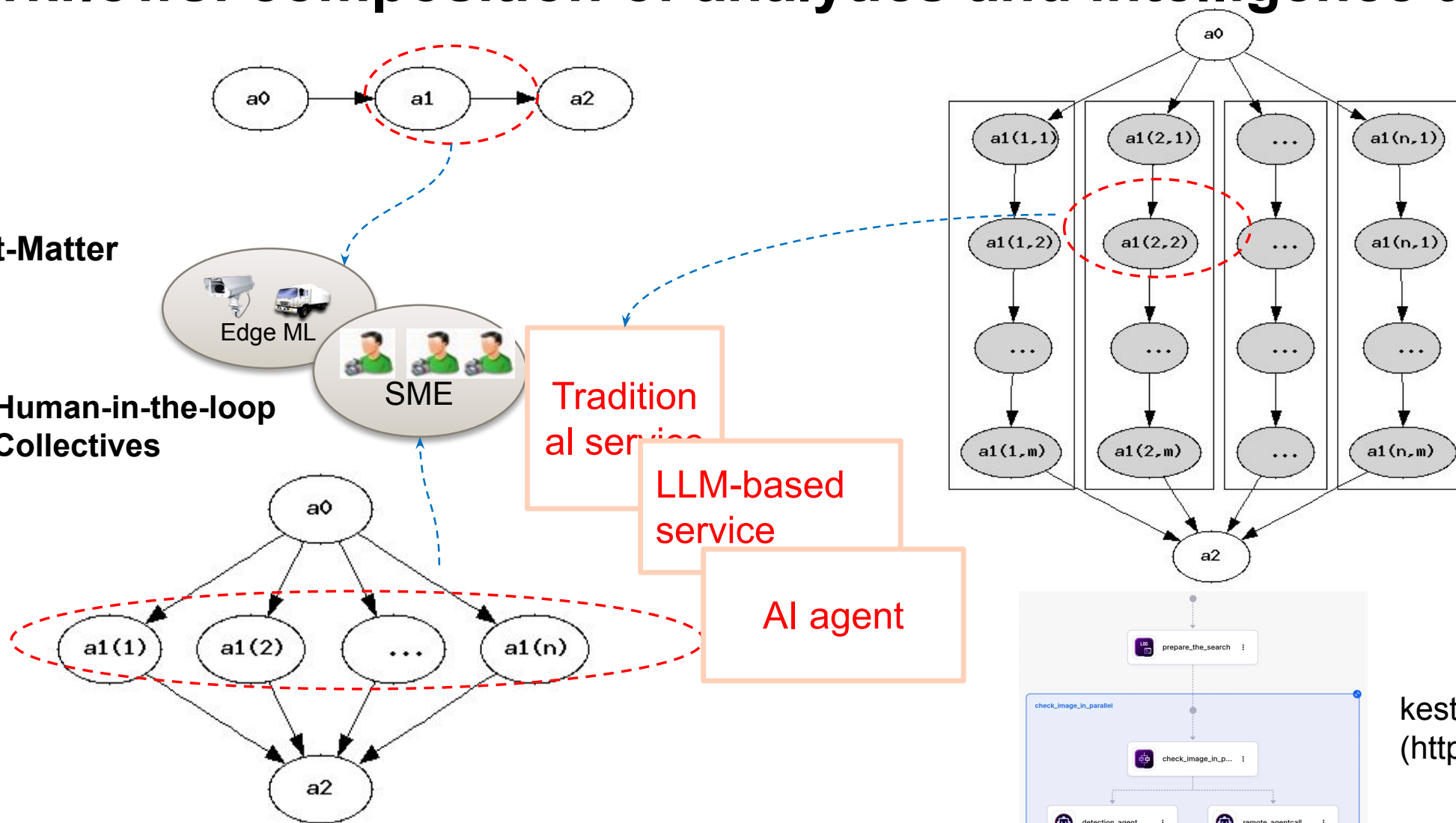


Invoke and control services to provision/change capabilities: utilize, add, remove, and change capabilities

Workflows: composition of analytics and intelligence tasks

SME:
Subject-Matter
Expert

**Human-in-the-loop
Collectives**



kestra UI
(<https://kestra.io/>)

Coordination workflow as code

- Coordination logic is implemented as code
 - hard wire programming code
- Commonly seen in emerging AI agent frameworks

Example: LangGraph

```
# Build workflow
orchestrator_worker_builder = StateGraph(State)

# Add the nodes
orchestrator_worker_builder.add_node("orchestrator", orchestrator)
orchestrator_worker_builder.add_node("llm_call", llm_call)
orchestrator_worker_builder.add_node("synthesizer", synthesizer)

# Add edges to connect nodes
orchestrator_worker_builder.add_edge(START, "orchestrator")
orchestrator_worker_builder.add_conditional_edges(
    "orchestrator", assign_workers, ["llm_call"]
)
orchestrator_worker_builder.add_edge("llm_call", "synthesizer")
orchestrator_worker_builder.add_edge("synthesizer", END)
```

Source: <https://docs.langchain.com/oss/python/langgraph/workflows-agents#orchestrator-worker>

Example: CrewAI

```
# Hierarchical crew
crew = Crew(
    agents=[manager, researcher, writer],
    tasks=[project_task],
    process=Process.hierarchical, # Manager coordinates everything
    manager_llm="gpt-4o", # Specify LLM for manager
    verbose=True
)
```

Source: <https://docs.crewai.com/v1.15.22/en/concepts/collaboration>

Tasks followed the chains of services

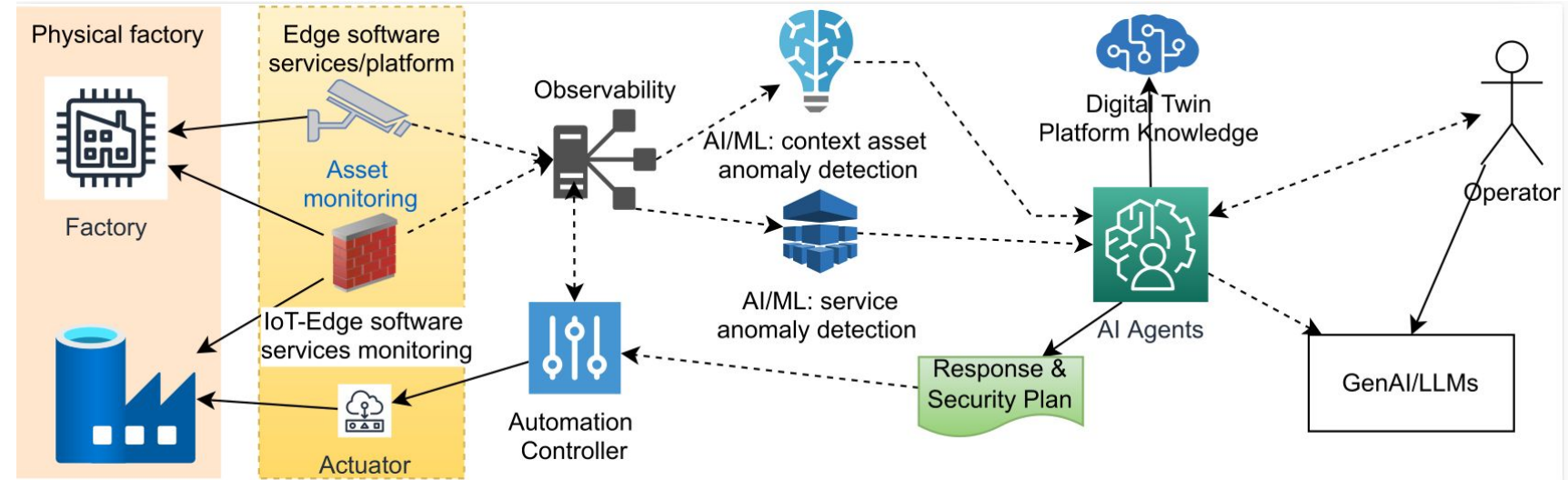
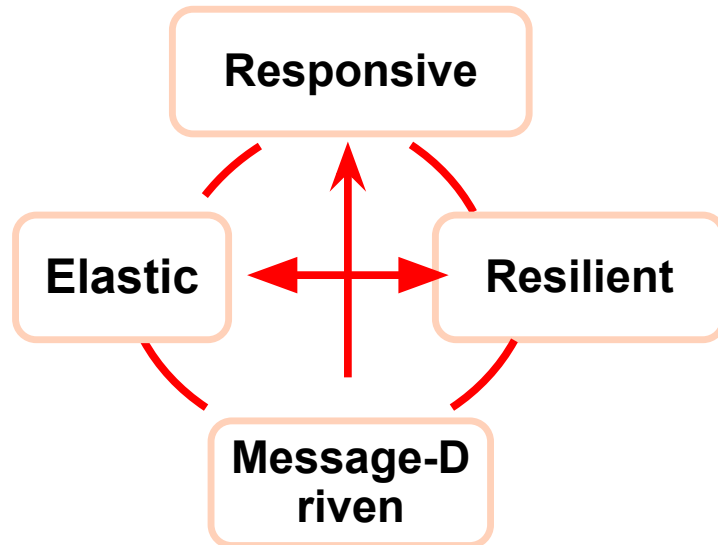


Figure source: Hong-Linh Truong, "New Frontiers in Service Engineering Analytics for Multi-Continuum Systems (FrontSEA)", working paper, 2025

- **Different services interacting via composable logics**
 - task descriptions and exchange topics/channels are used for delivering tasks
 - services would bid and perform selected tasks
 - results are also exchanged via topics/channels
- **Highly asynchronous and complex observability**

Coordination with the choreography/reactiveness style (1)

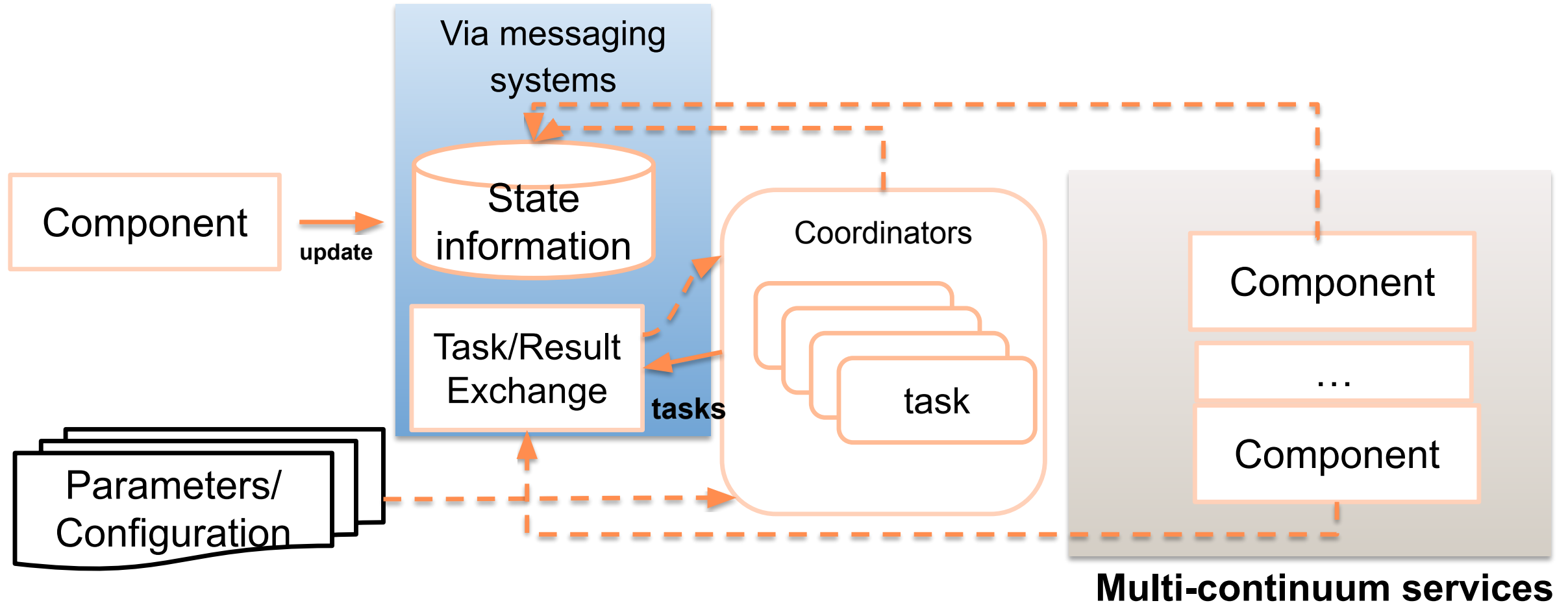
Rely on principles of reactive systems



Source: <https://www.reactivemanifesto.org/>

- Messaging systems for exchanges of tasks and results
 - Kafka, RabbitMQ, ZeroMQ, Amazon SQS, ...
 - types of messages and semantics must be defined clearly
- Triggers and controls
 - the worker model:
 - services act as job workers listening messages to trigger (remote) functions/tasks

Coordination with the choreography/reactiveness style (2)



Different coordination architectures (where)

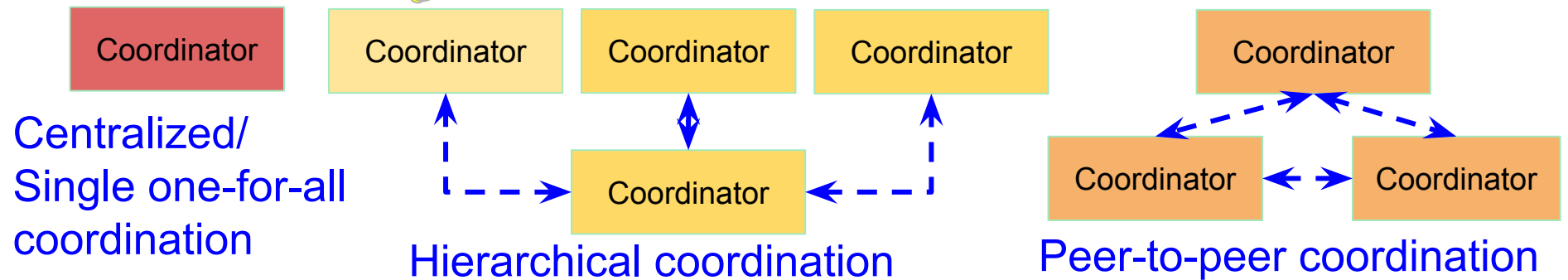
Coordination style

Workflow Style

Reactive Style

which one?

Possible architecture designs (Where)



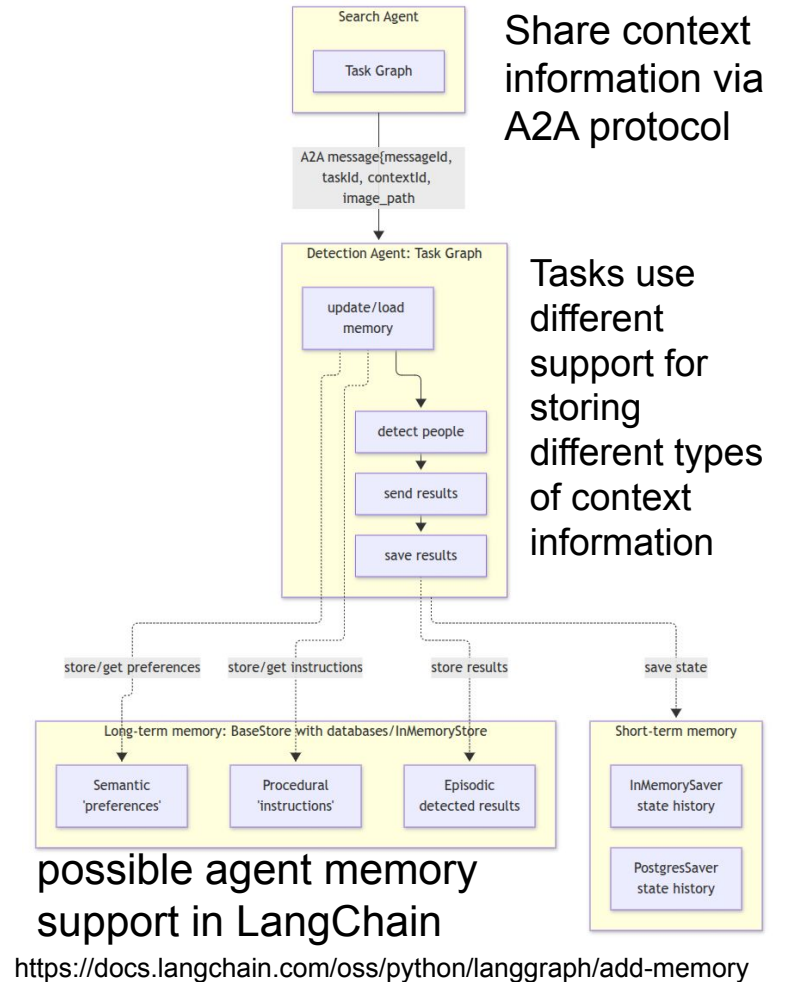
Instructions/skills: AI agents and humans

- We can have skills and instructions for agents and humans
 - AI Agents: there is `SKILL.md` but what about human services?
 - but will they actually have the declared skills and follow the claim?
- Capability and profiling description
 - maybe unclear with LLM-based services, AI agents and humans
 - traditional AI/ML Services: fixed constraints
- AI Agents and humans
 - non deterministic performance and hallucination
 - high risk of trustworthiness, sovereignty and performance guarantees

Sharing context information between the coordinator and invoked services

- 3 common approaches
 - shared storage
 - e.g., can be controlled by the coordinator
 - embedded into messages/task information exchanged
 - message is heavy, but maintain “microservices” principles
 - up to the coordinator and services to find their ways
- Using cross-reference id (e.g., task id and context id) for getting context information
- Do not confuse with AI agent memory
 - Agent memory is specific to Agent frameworks
 - we can use different techniques for managing context information

Example



Speed influencing coordination strategies

	Humans	AI Agents	LLM-based services	Traditional AI/ML services
Decisiveness (how quickly to make a decision)	slow	fast	fast	fast
Reactiveness (how quickly to respond to change)	yes, very high	yes, high	not really (done outside the service)	not really (done outside the service)
Performance (how quickly to finish the work)	slow	fast	fast	fast

**fast: dependent on the infrastructure capabilities*

When does a type of services become a bottleneck? E.g., humans for quality control or AI agents for decision making

Observability and assurance during runtime coordination

- Observability must be done for tasks to be executed
 - provide also detailed trajectory-level paths
- Have clear constraints on sovereignty and performance
 - constraints on data and compute sovereignty
 - constraints on performance (part of SLAs - time, tokens, data quality)
- Runtime policies and enforcement for constraints
- Invoking human-in-the-loop for assurance is tricky
 - not the same as humans for performing tasks

Patterns

- Patterns specify micro-level styles + coordination protocols/mechanisms
- Some patterns
 - Random, fixed manual
 - Round Robin
 - Fan-out Fan-in
 - Dynamic routing
 - Workflow/Fix pipeline
 - Decentralized backboard, content-based dispatching
- Coordination
 - we focus on runtime coordination for task completion, not software integration design (e.g., in the previous lecture)

Try to analyze **pros and cons** when using these patterns in

- a coordinator within single service
- a centralized coordinator for many services
- coordinators within services working together in a peer-to-peer fashion

Prepare your course project

- No study log but prepare for your course project
- Topics
 - try to think and propose your own ideas based on your reading, your work, and open questions during the course discussion
 - if not possible, you can select one from the pre-defined topic
- Presentation/discussion
 - each of you will have 10 minutes max

A!

—

Kiitos
aalto.fi